

NAME :G.AKSHAYA

COURSE CODE : CSA0619

REG NO: 192521156

COURSE NAME : DESIGN AND

ANALYSIS OF ALGORITHMS

CO3 - ASSESSMENT TOOL – 02

Q2. Case Study: Real-Time Ranking System QuickSort

Aim

To analyze the application of the Quick Sort algorithm in a real-time ranking system for efficiently sorting dynamically updated scores, understand its divide-and-conquer strategy, evaluate its performance, identify issues such as pivot selection and worst-case complexity, and propose optimized solutions.

Problem Statement

A real-time ranking system continuously updates user scores after every event or transaction. To display the latest rankings instantly, the system requires a fast and efficient sorting algorithm. Quick Sort is suitable because of its high average-case performance and divide-and-conquer approach.

Objective

- Sort ranking scores efficiently.
- Update leaderboard rankings in real time.
- Reduce sorting and refresh time.
- Improve performance using optimized pivot selection.
- Analyze time and space complexity..

Working of Divide and Conquer

Divide

Select a pivot element and divide the scores into two groups.

Example:

Scores:

[95, 70, 88, 60, 99, 75]

Pivot = **88**

Left:

70, 60, 75

Right:

95, 99

Conquer

Recursively sort both partitions.

Left:

60, 70, 75

Right:
95, 99

Combine

Final Sorted Rankings:

60 0 75 88 95 99

Issues in Quick Sort

1. Pivot Selection

Choosing the first or last element repeatedly may create highly unbalanced partitions.

Example:

Already Sorted Scores:

0

70

75

88

95

99

Produces:

1 element | 5 elements

↓

1 element | 4 elements

↓

1 element | 3 elements

This results in **$O(n^2)$** time complexity.

2. Worst-Case Performance

Occurs when:

- Array is already sorted.
- Array is reverse sorted.
- Poor pivot selection.
- Large number of duplicate values.

Worst-case complexity: $O(n^2)$

3. Recursive Calls

Large datasets increase recursion depth and consume additional memory.

Randomized Pivot Selection

Optimized Solution

Randomized Pivot Selection

Instead of choosing the first or last element, randomly select the pivot.

Advantages

- Avoids worst-case partitions.
- Produces balanced partitions.
- Improves average sorting performance.
- Suitable for dynamic real-time ranking systems.
- Faster leaderboard updates.

Algorithm

1. Start.
2. Read the ranking scores.
3. Select a random pivot.
4. Partition the scores into:
 - Less than pivot
 - Equal to pivot
 - Greater than pivot
5. Recursively sort the left and right partitions.
6. Combine all partitions.
7. Display the updated sorted rankings.
8. Stop.

SOURCE CODE

```
main.py
1 import random
2 def partition(arr, low, high):
3     # Randomized Pivot Selection
4     pivot_index = random.randint(low, high)
5     arr[pivot_index], arr[high] = arr[high], arr[pivot_index]
6     pivot = arr[high]
7     i = low - 1
8     for j in range(low, high):
9         if arr[j] <= pivot:
10             i += 1
11             arr[i], arr[j] = arr[j], arr[i]
12     arr[i + 1], arr[high] = arr[high], arr[i + 1]
13     return i + 1
14 def quick_sort(arr, low, high):
15     if low < high:
16         pi = partition(arr, low, high)
17         quick_sort(arr, low, pi - 1)
18         quick_sort(arr, pi + 1, high)
19 # Main Program
20 scores = [95, 70, 88, 60, 99, 75]
21 print("Player Scores Before Sorting:")
22 print(scores)
23 quick_sort(scores, 0, len(scores) - 1)
24 print("\nLeaderboard After Quick Sort:")
25 print(scores)
```

OUTPUT

```
Output
Player Scores Before Sorting:
[95, 70, 88, 60, 99, 75]

Leaderboard After Quick Sort:
[60, 70, 75, 88, 95, 99]

=== Code Execution Successful ===
```

Complexity Analysis

Case	Time Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$

Space Complexity:

$O(\log n)$ (Average recursion stack)

Result

The Quick Sort algorithm efficiently sorts dynamically updated scores in a real-time ranking system using the divide-and-conquer technique. It provides an average time complexity of $O(n \log n)$, making it suitable for fast leaderboard updates. By using randomized pivot selection, the possibility of worst-case performance is greatly reduced, resulting in improved efficiency, balanced partitions, and faster ranking updates.